

Runtime Inspector

Un nuovo modo di sviluppare interfacce React Native

Introduzione

Runtime Inspector nasce da un problema reale: sviluppare e rifinire animazioni, micro-interazioni ed effetti grafici in React Native è molto più lento di quanto dovrebbe essere. Oggi disponiamo di strumenti potenti come Reanimated, Skia, shader AGSL, Metal e moduli nativi, ma il processo di ottimizzazione continua a basarsi su modifiche manuali al codice, Fast Refresh e tentativi ripetuti.

Il problema

Il ciclo classico è: modificare un valore, salvare, attendere il refresh, riprodurre l'animazione e ripetere. Questo approccio diventa inefficiente quando entrano in gioco spring, easing, blur, shader, gesture e decine di parametri che devono essere percepiti visivamente più che calcolati.

Perché un pannello esterno

Inserire un pannello di debug direttamente nell'app riduce lo spazio disponibile e rende difficile osservare l'animazione nel suo contesto reale. L'idea è quindi separare completamente l'applicazione dal pannello di controllo.

L'idea

L'app React Native continua ad eseguire normalmente l'interfaccia. Un'applicazione esterna (Web, Desktop, VS Code o DevTools) costruisce automaticamente un pannello di controllo leggendo uno schema dichiarativo esposto dall'app. Lo sviluppatore descrive solo i controlli disponibili; non costruisce manualmente l'interfaccia.

Protocollo dichiarativo

Il cuore del progetto non è il pannello ma un protocollo condiviso. L'app dichiara gruppi e controlli (slider, toggle, editor di curve, colori, spring ecc.). Il client interpreta lo schema e genera dinamicamente l'interfaccia. Ogni modifica produce una patch che viene inviata al runtime e applicata immediatamente.

Architettura

Il sistema è composto da quattro elementi: Runtime React Native, Runtime SDK, Protocollo Runtime Inspector e Client (Panel). Il trasporto iniziale avviene tramite WebSocket locale, ma il protocollo rimane indipendente dal mezzo di comunicazione.

Flusso operativo

L'app registra i pannelli, il client scopre il runtime, riceve lo schema, costruisce la UI, invia patch in tempo reale e il runtime aggiorna SharedValue o altri binding senza richiedere re-render dell'interfaccia.

Preset

Quando il risultato è soddisfacente, il pannello esporta un preset serializzabile (JSON o TypeScript) che diventa parte del codice sorgente. Il pannello non sostituisce il codice: accelera il processo di definizione dei valori corretti.

Visione

Runtime Inspector vuole definire uno standard aperto per descrivere e modificare il comportamento di un'applicazione in esecuzione. React Native rappresenta il primo runtime supportato, ma l'architettura è progettata per poter estendersi a SwiftUI, Jetpack Compose, Flutter e Web mantenendo lo stesso protocollo.